

1. String-to-Function Mechanics (Passing Strings)

When passing an array or a string into a function, you are not copying the actual values. Instead, you pass the memory address of its very first element (**the base address**).

Function Prototype Notation Options:

void MyFunction(char *str); <---- **Pointer Notation (Standard & Preferred)**

void MyFunction(char str[]); <---- **Array Notation (Alternative)**

Both syntaxes are compiled identically under the hood. However, pointer notation (char *) explicitly signals that the function is receiving a mutable reference tracking a character sequence block in memory.

Custom strcmp Logic Pipeline

Here is how we accept raw pointers to run a character-by-character validation loop until a mismatch or a termination flag (\0) breaks the cycle:

```
#include <stdio.h>
void custom_strcmp(char *str1, char *str2); ----> Prototype
int main()
{
    char str1[30];
    char str2[30];

    printf("Enter string str1: ");
    fgets(str1, 30, stdin);
    printf("Enter string str2: ");
    fgets(str2, 30, stdin);

    custom_strcmp(str1, str2); ----> Passes base addresses
    return 0;
}

void custom_strcmp(char *str1, char *str2)
{
    int flag = 0;
    int i = 0;

    for (i = 0; str1[i] != '\0' && str2[i] != '\0'; i++) ----> Iterate until either string encounters a null terminator
    {
        if (str1[i] != str2[i])
        {
            flag = 1;
            break;
        }
    }
}
```

```

    }
}

if (flag == 1)
{
    printf("Strings are not same\n");
}
else
{
    printf("Strings are same\n");
}
}

```

2. Multi-String Handling (2-D Arrays)

To store multiple independent names or words, we utilize a **Two-Dimensional Character Array**

```
char str[6][8] = {"Gaurav", "Suraj", "Chandu", "Kishor", "Kannu", "Deepu"};
```

[6] (Rows): Defines the total number of distinct strings being stored

[8] (Columns): Defines the maximum available byte length allocated for each string[cite: 9].

Contiguous Memory Grid Visual Matrix

Row Index	Character Layout Columns 	Base Ptr Address
str[0] →	['G']['a']['u']['r']['a']['v']['\0'][]	65454
str[1] →	['S']['u']['r']['a']['j']['\0'][]	65462
str[2] →	['C']['h']['a']['n']['d']['u']['\0'][]	65470

Pointer Interpretation: str[0] provides the base memory pointer for the 0th string [str[i][j]] explicitly targets the **j** single character situated within the **i** string entry

How to Swap Strings Inside a 2-D Grid

You can swap the positions of two strings using two structural methodologies

Character-by-Character Loop: Swap individual elements index-by-index using a holding variable `ch = s1[i]; s1[i] = s2[i]; s2[i] = ch;`

Bulk Buffer Copy (strcpy): Duplicate data strings using a dedicated proxy array buffer :

```
char temp[20]; ----> Temporary buffer string block
strcpy(temp, str[0]);
strcpy(str[0], str[1]);
strcpy(str[1], temp);
```

Reversing Strings (Standard & Custom)

Approach 1: Non-Standard Library Call (strrev)

strrev() is a fast, **non-standard** utility provided by some compiler setups (like Turbo C) to flip a string in-place

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[30] = "Gaurav Kandpal";
    printf("String after reversing: %s\n", strrev(str));
    return 0;
}
```

Approach 2: Two-Pointer Swap Algorithm (No Library Dependency)

To reverse a string manually without using library helpers, we loop up to $\text{Length}/2$. We use a symmetric index mapping formula to swap the front index with its corresponding back index.

$$\text{Opposite Target Index} = \text{Length} - 1 - i$$

Front Pointer [i] $\longrightarrow$ ['G']['a']['u']['r']['a']['v'] $\longleftarrow$ [Length - 1 - i] Back Pointer

$\uparrow$ $\uparrow$
Swap Elements

In-Place Reverse Implementation

```
#include <stdio.h>
#include <string.h>
int main()
{
    char ch;
    char str[30] = "Gaurav Kandpal";
```

```

int L = strlen(str); ----> L = 14
for (int i = 0; i < L / 2; i++) ----> Loop through exactly half of the string length
{
    ch = str[i]; ----> Store front character
    str[i] = str[L - 1 - i]; ----> Move back character to front
    str[L - 1 - i] = ch; ----> Assign saved front character to back
}
printf("String after reversing: %s\n", str); ----> Output: lapdnak varuaG
return 0;
}

```

4. Advanced Challenge: Reversing the Order of Words

When building a sentence parser, you may want to reverse the words themselves while keeping their inner spelling intact (e.g., turning "You are Good Guy" into "Guy Good are You")

Algorithm Strategy Flowchart

Step 1: Reverse the entire string completely.

"You are Good Guy" → "yuG doof era uoY"

Step 2: Loop through and locate space boundaries (' ') or the null terminator ('\0'). Whenever a word boundary is targeted, reverse that isolated word segment in-place.

"yuG" → "Guy" | "doof" → "Good" | "era" → "are" | "uoY" → "You"

Final Result: "Guy Good are You"

Sentence Word Sorter Blueprint

```

#include <stdio.h>
#include <string.h>
void reverse_segment(char *str, int start, int end); void reverse_words(char *str);
int main()
{
    char str[100] = "You are Good Guy";

    reverse_words(str);
    printf("String after reversing: %s\n", str); ---> Output: Guy Good are You
    return 0;
}
void reverse_segment(char *str, int start, int end) ----> Swaps characters within an
{                                                    isolated window region
    char temp;

```

```

while (start < end)
{
    temp = str[start];
    str[start] = str[end];
    str[end] = temp;
    start++;
    end--;
}
}

void reverse_words(char *str)
{
    int len = strlen(str);
    reverse_segment(str, 0, len - 1); ----> Step 1: Reverse the whole string
    int word_start = 0; ----> Step 2: Reverse individual words in-place

    for (int i = 0; i <= len; i++)
    {
        if (str[i] == ' ' || str[i] == '\0') -----> Look for spaces or the sentence end marker
        {
            reverse_segment(str, word_start, i - 1);
            word_start = i + 1; ----> Update start tracker to the next word
        }
    }
}

```

5. Letter Case Transformations (strupr / strlwr)

C provides string library helpers to convert character case sets

strupr(str); Converts all lowercase letters to upper case

strlwr(str); Converts all uppercase letters to lower case

The Mathematical Mechanics: ASCII Offset Shift

Every character corresponds to an underlying numerical value.

'A' is mapped to **65**, while 'a' is mapped to **97**.

The constant distance offset between upper and lower case representations is exactly **32**

Lower-to-Upper Conversion : $\text{str}[i] = \text{str}[i] - 32$

Upper-to-Lower Conversion : $\text{str}[i] = \text{str}[i] + 32$

Custom Case Converter Implementation

```
#include <stdio.h>
void custom_strupr(char *str);
void custom_strlwr(char *str);
int main()
{
    char text[50] = "Gaurav Kandpal";

    custom_strupr(text);
    printf("String in upper case: %s\n", text); ----> Output: GAURAV KANDPAL

    custom_strlwr(text); //[cite: 9]
    printf("String in lower case: %s\n", text); ----> Output: gaurav kandpal
    return 0;
}

void custom_strupr(char *str) ----> Converts lowercase characters to uppercase
{
    int i = 0;
    while (str[i] != '\0')
    {
        if (str[i] >= 'a' && str[i] <= 'z')
        {
            str[i] = str[i] - 32; ----> Shifts down to uppercase memory values
        }
        i++;
    }
}

void custom_strlwr(char *str) ----> Converts uppercase characters to lowercase
{
    int i = 0;
    while (str[i] != '\0')
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
        {
            str[i] = str[i] + 32; ----> Shifts up to lowercase memory values
        }
        i++;
    }
}
```